

VC_TIMING

Overview

vc_timing monitors timing relationship between two single-bit signals using a set of counters. The monitoring counter values provide the user with a convenient means to constrain or cover any timing relationships between two signals. By dividing the protocol constraints into timing constraints between interdependent signal pairs, the **vc_timing** component can be successfully used for a complete timing verification of the protocol.

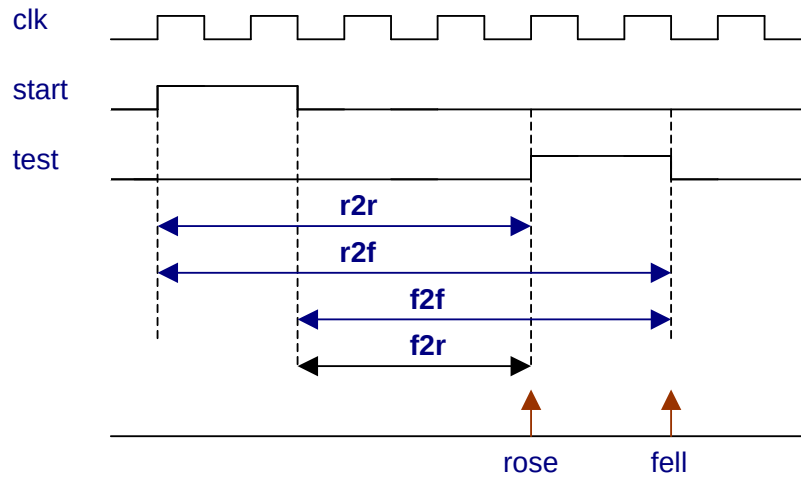
Syntax

vc_timing #(CWIDTH) InstName (clk, reset_n, start, test)

Parameters	
CWIDTH	Counters bit width. Default: 8
Interface	
Clk	Clock input
reset_n	Active-low reset
Start	1-bit signal defining start event
Test	1-bit signal defining test event
Access Registers	
Distance	Monitors the actual distance in clock cycles between the start and test events. Can be used for statistics monitoring with a vc_stat component.
Access Events	
rose, fell, changed	Monitor transitions of the test signal. Must be used as timing controls for external "always" statements, which sets timing constraints.
Access Registers	
r2r	Rise-to-Rise value (in clock cycles). Sampled using a "rose" event.
r2f	Rise-to-Fall value (in clock cycles). Sampled using a "fell" event.
f2r	Fall-to-Rise value (in clock cycles). Sampled using a "rose" event.
f2f	Fall-to-Fall value (in clock cycles). Sampled using a "fell" event.

Usage

vc_timing allows users to dynamically monitor any possible timing relationships between two given signals. The following picture illustrates the vc_timing functionality:



In order to set constraint, user validates the values of counters upon events: rose, fell and changed. The following code sets timing constraints between the req and ack signals:

```
//synopsys synthesis_off
vc_timing req_ack (clk, 1'b1, req, ack);

always @(req_ack.rose) begin
    if (r2r < 3 || r2r > 8) $display("Error: r2r out of time frame");
    if (f2r < 1 || f2r > 6) $display("Error: f2r out of time frame");
end

always @(req_ack.fell) begin
    if (r2f < 5 || r2f > 10) $display("Error: r2f out of time frame");
    if (f2f < 3 || f2f > 8) $display("Error: f2f out of time frame");
end
//synopsys synthesis_on
```

vc_timing also allows to connect the same signal to both the start and test inputs. In this case, the component will monitor the pulse widths of a signal, for example:

```
vc_timing req_t (clk, 1'b1, req, req);

always @(req_t.rose) begin
    if (r2r < 3 || r2r > 8) $display("Error: req r2r out of time frame");
    if (r2f < 2) $display("Error: req pulse is too narrow");
end
```

Example

```
module test;
reg a, b, clk;

initial begin
    $dumpvars;
    clk = 0;
    a = 0; b=0;
    #20 a = 1;
    #20 b = 1;
    #20 a = 0;
    #20 b = 0;
    #20 a = 1;
    b = 1;
    #40 $finish;
end

always #10 clk = ~clk;
vc_timing a_b (clk, 1'b1, a, b);
always @(a_b.rose)
    $display("[%0t]: b rose, r2r = %0d, f2r = %0d", $time, a_b.r2r, a_b.f2r);
always @(a_b.fell) begin
    $display("[%0t]: b fell, r2f = %0d, f2f = %0d", $time, a_b.r2f, a_b.f2f);
    if (a_b.r2f < 20)
        $display("[%0t] <%m> Error: a_b rise-to-fall time (%0d) less than 20!",
            $time, a_b.r2f);
end

endmodule
```